

Template Method Pattern

Category: Behavioural Design Pattern

1. The Problem

Consider different data parsers — CSV, JSON, XML. Each parser follows the same steps: open file, parse data, close file. Without the Template Method Pattern, the common steps end up duplicated in every parser class.

```
class CSVParserIn {
    public void parse(){
        openFile();
        //CSV Specific Parsing Logic
        System.out.println("Parsing a CSV File");
        closeFile();
    }
    private void openFile(){
        System.out.println("Opening File");
    }
    private void closeFile(){
        System.out.println("Closing File");
    }
}

class JSONParserIn {
    public void parse(){
        openFile();
        //JSON Specific Parsing Logic
        System.out.println("Parsing a JSON File");
        closeFile();
    }
    private void openFile(){
        System.out.println("Opening File");
    }
    private void closeFile(){
        System.out.println("Closing File");
    }
}

public class WithoutTemplatePattern {
    public static void main(String[] args) {
        CSVParserIn csvParser = new CSVParserIn();
        JSONParserIn jsonParser = new JSONParserIn();
        csvParser.parse();
        jsonParser.parse();
    }
}
```

Issues: - **Code duplication** — `openFile()` and `closeFile()` are duplicated identically in both parsers. - Any change to the common logic (e.g. how files are opened/closed) needs to be made in every parser class, violating the **DRY (Don't Repeat Yourself)** principle.

2. The Solution — Template Method Pattern

Intent: Define the skeleton of an algorithm in a base class, and let subclasses override only the specific steps that need to vary — while the overall structure/sequence stays fixed.

- The common steps (`openFile()` , `closeFile()`) live once in the base class.
- The step that actually differs per format (`parseData()`) is left abstract, to be implemented by each subclass.

3. Key Idea

- **Invert control of the algorithm’s structure** — the base class owns the fixed sequence of steps (the “template”), and calls out to subclass-specific steps at the right points, rather than each subclass independently reimplementing the whole sequence.
- The overall algorithm method (`parse()`) is marked **final** so subclasses can’t change the *order* or *structure* of steps — they can only plug in behavior for the steps explicitly left open (`parseData()`).
- This is a direct application of **DRY** and the **Hollywood Principle** (“don’t call us, we’ll call you”) — subclasses don’t drive the flow; the base class does, and it calls into the subclass when needed.

4. Variants

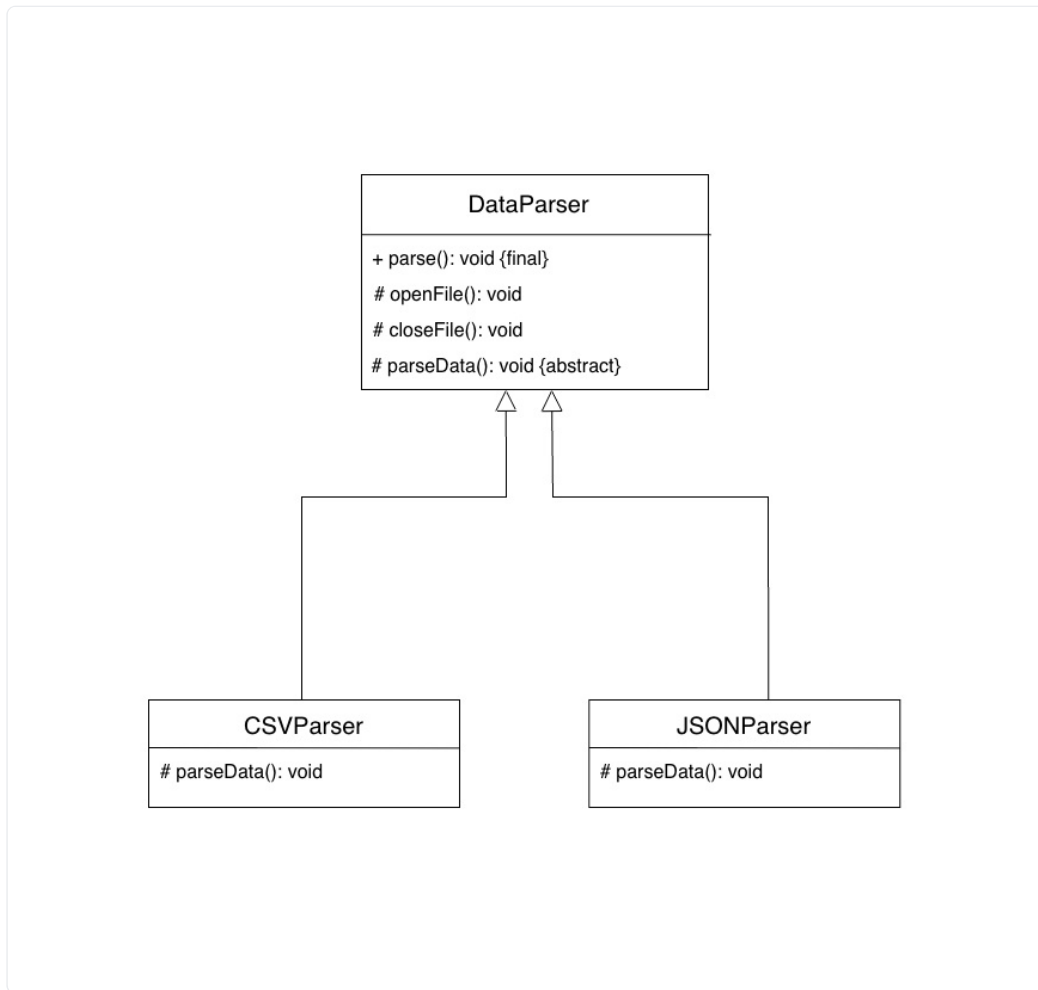
Not really applicable here — this is the standard Template Method structure. A common extension (not shown in this code) is **hook methods**: optional steps with a default (often empty) implementation in the base class that subclasses *may* override, as opposed to abstract steps that subclasses *must* override (like `parseData()` here).

5. Structure / Participants

Role	Responsibility	Example (from this exercise)
Abstract Class	Defines the algorithm’s skeleton (<code>parse()</code>), including calls to both concrete and abstract steps	<code>DataParser</code>
Concrete Subclass	Implements/overrides only the specific step(s) left abstract by the base class	<code>CSVParser</code> , <code>JSONParser</code>

Relationship type: Inheritance — concrete parsers *extend* `DataParser` and inherit its fixed `parse()` skeleton, while providing their own implementation of `parseData()` .

Diagram



Template Pattern UML diagram

`DataParser` (abstract class) defines `parse()` as `final` — the fixed skeleton — along with concrete helper steps `openFile()` and `closeFile()`, and one abstract step, `parseData()`. `CSVParser` and `JSONParser` extend `DataParser` and each override only `parseData()`, plugging in their own format-specific logic while inheriting the same open → parse → close sequence.

6. Code — Full Implementation

```
abstract class DataParser {

    public final void parse(){
        openFile();
        parseData();
        closeFile();
    }

    protected void openFile(){
        System.out.println("Opening File");
    }

    protected void closeFile(){
        System.out.println("Closing File");
    }

    protected abstract void parseData();
}
```

```

    }

    class CSVParser extends DataParser {
        @Override
        protected void parseData() {
            System.out.println("Parsing CSV Data");
        }
    }

    class JSONParser extends DataParser {
        @Override
        protected void parseData() {
            System.out.println("Parsing JSON Data");
        }
    }

    public class TemplatePattern {
        public static void main(String[] args) {
            DataParser csvParser = new CSVParser();
            DataParser jsonParser = new JSONParser();
            csvParser.parse();
            jsonParser.parse();
        }
    }
}

```

What changed vs. the “without pattern” version: - `openFile()` / `closeFile()` duplicated in both `CSVParserIn` and `JSONParserIn` → moved once into the abstract `DataParser` base class. - Each parser had its own full `parse()` method reimplementing the whole sequence → `parse()` now lives once in `DataParser`, marked `final`, and is inherited unchanged by every subclass. - Format-specific logic was inline inside each class’s `parse()` → isolated into a single `parseData()` method that each subclass must override. - Parsers were independent, unrelated classes → `CSVParser` / `JSONParser` now extend `DataParser`, establishing an explicit inheritance relationship.

7. Benefits

- **Code Reuse** — common logic (`openFile()` , `closeFile()` , and the overall flow) lives once in the parent class.
- **Flexibility** — subclasses can vary the specific steps that need to differ, without touching the overall structure.
- **Consistency** — the high-level structure/sequence of the algorithm stays identical across all subclasses, since `parse()` is `final`.

8. Drawbacks / Things to Watch Out For

- Relies on **inheritance**, which is a tighter coupling than composition-based patterns (like Strategy) — subclasses are locked into the parent’s structure.
- Can lead to a rigid hierarchy if many small variations are needed — every variation becomes a subclass.
- If the base class’s fixed steps ever need to vary too, the “closed” parts of the template can become a limitation, requiring parallel hook methods or refactoring.

9. Real-World Use Cases

- **UI Frameworks** — rendering a UI element follows fixed steps (initialize, draw, finish), with each element defining its own drawing details.
- **Document Processing** — a framework defines the skeleton for reading, processing, and saving documents, while specific formats (Word, PDF) supply their own processing logic.
- **Game Development** — a game loop (initialize, update, render) defined in a base class, with individual games implementing their own update/render logic.

10. Interview Questions & Answers

Q1. What is the Template Method Pattern? A behavioural design pattern that defines the fixed skeleton of an algorithm in a base class, deferring only specific steps to subclasses — so the overall structure stays consistent while individual steps can vary.

Q2. What problem does it solve? It eliminates duplication of common algorithm steps across classes that otherwise each reimplement the same overall sequence, keeping the code DRY and ensuring the structure of the algorithm can't drift between implementations.

Q3. What are its main components/participants? - **Abstract Class** — defines the algorithm's skeleton, including calls to both concrete helper methods and abstract steps - **Concrete Subclass** — implements only the abstract step(s) left open by the base class

Q4. Why is the template method (e.g. `parse()`) typically declared `final`? To prevent subclasses from overriding the overall structure/sequence of the algorithm — they should only be able to customize the specific steps the base class explicitly leaves open (like `parseData()`), not change the order or presence of steps.

Q5. What's the difference between an abstract step and a "hook" in the Template Method Pattern? An abstract step (like `parseData()` here) has no default implementation and *must* be overridden by every subclass. A hook is a step with a default (often empty or no-op) implementation in the base class that subclasses *may* optionally override if they need to customize that part — otherwise the default behavior is used as-is.

Q6. How does the Template Method Pattern differ from the Strategy Pattern? Both let part of a behavior vary, but Template Method uses **inheritance** — subclasses override specific steps of an algorithm whose overall structure lives in the parent class. Strategy uses **composition** — the varying behavior is a completely separate, swappable object injected into a context, with no inheritance relationship between the context and the behavior.

Q7. What are the benefits? Code reuse of common steps, flexibility to vary specific steps per subclass, and guaranteed consistency of the algorithm's overall structure across all implementations.

Q8. What are the drawbacks? Tighter coupling via inheritance rather than composition, and a hierarchy that can grow rigid or unwieldy if many small variations are needed.

Q9. Can you give real-world examples? UI element rendering pipelines, document processing frameworks (reading/processing/saving different file formats), and game loops (initialize/update/render) implemented per-game on a shared base structure.